PROJECT IMPLEMENTATION REPORT

PetStore

A three-service shop for pets, with the catalog, orders, images, and secrets running on Azure.

Resource group · Petstore-Resource-Group
Subscription · Petstore subscription
Date · 22 October 2026
Evidence · Screenshots captured from the running UI and the Azure portal

Product service · Order service · Pet service · React storefront · PostgreSQL · Blob Storage · Key Vault · Container Registry

1. What this project covers

PetStore is a small shop in which every pet is a product for sale. A visitor can browse, filter, and open a pet without signing in. Buying, order history, and administration are signed-in flows. The browser talks to two APIs. A third service shares the same database and the same secret source.

Area	What is in place	Shown by
Storefront	Catalog, category chips, details, cart drawer, checkout	Figure 1, Figure 2
Customer account	Separate shop session, order history, logout	Figure 3
Administration	Categories, pets with a required photo, all orders, delete	Figure 4, Figure 5, Figure 6
Database	Azure Database for PostgreSQL Flexible Server, database `petstore`	Figure 7, Figure 8
Images	Azure Blob Storage account `petstore`, container `pet-container`	Figure 9, Figure 10
Secrets	Key Vault `db-credentials-petstore`. No database password, connection string, or account key in application YAML	Figure 11, Figure 12
Container images	Azure Container Registry `petstoreazurecontainerregistry`. GitHub Actions pushes four images on every push to `main`	Figure 13, Figure 14, Figure 15, Figure 16

product-service

Port 8081. Pets for sale, categories, and image upload. Reads the database username and password, plus the blob connection string and account key, from Key Vault.

order-service

Port 8083. Register, login, checkout, and orders. Issues the JWT. Makes sure the admin account exists. Reads the database username and password from Key Vault.

pet-service

Port 8082. Pet API on the same database. Reads the database username and password from the same vault. The shop UI sells through the product service.

The React application runs on port 3000. Swagger is available at `http://localhost:8081/swagger-ui.html` and `http://localhost:8083/swagger-ui.html`.

2. How the pieces fit

The browser keeps the cart in local storage. Login is issued by the order service. The product service trusts the same JWT. Shop and admin sessions are stored separately, so signing in as a customer does not sign the administrator out.

Caller	Calls	For
Storefront and admin UI	product-service `:8081`	Categories, pets, photos
Storefront and admin UI	order-service `:8083`	Register, login, checkout, orders
All three services	Key Vault `db-credentials-petstore`	Database username and password
product-service	Blob container `pet-container`	Upload, replace, and delete pet photos
All three services	PostgreSQL `petstore-server-name`	Database `petstore`
GitHub Actions on `main`	Container Registry `petstoreazurecontainerregistry`	Build and store the four application images

Application accounts

These are shop accounts stored in the database. They are not the Azure or database secrets.

Username	Password	Role
demo	demo123	USER
admin	admin123	ADMIN

> On startup, Hibernate creates any missing tables (`ddl-auto: update`). The order service creates the admin account when that username is missing. It does not reset an existing admin password.

3. Storefront

The public shop is the home page. There is no login wall. Category chips are built from the pets already loaded, so a guest can filter by All, Cat, Dog, or Parrot. Each card shows the photo, category, name, description, and price, with Details and Add to cart. A new pet cannot be saved without an image, which is why every card in the catalog has a photograph.

Add to cart asks a guest to sign in. For a signed-in shopper it opens the cart drawer on the same page. Checkout needs a shop login, then clears the cart and opens that customer's orders.

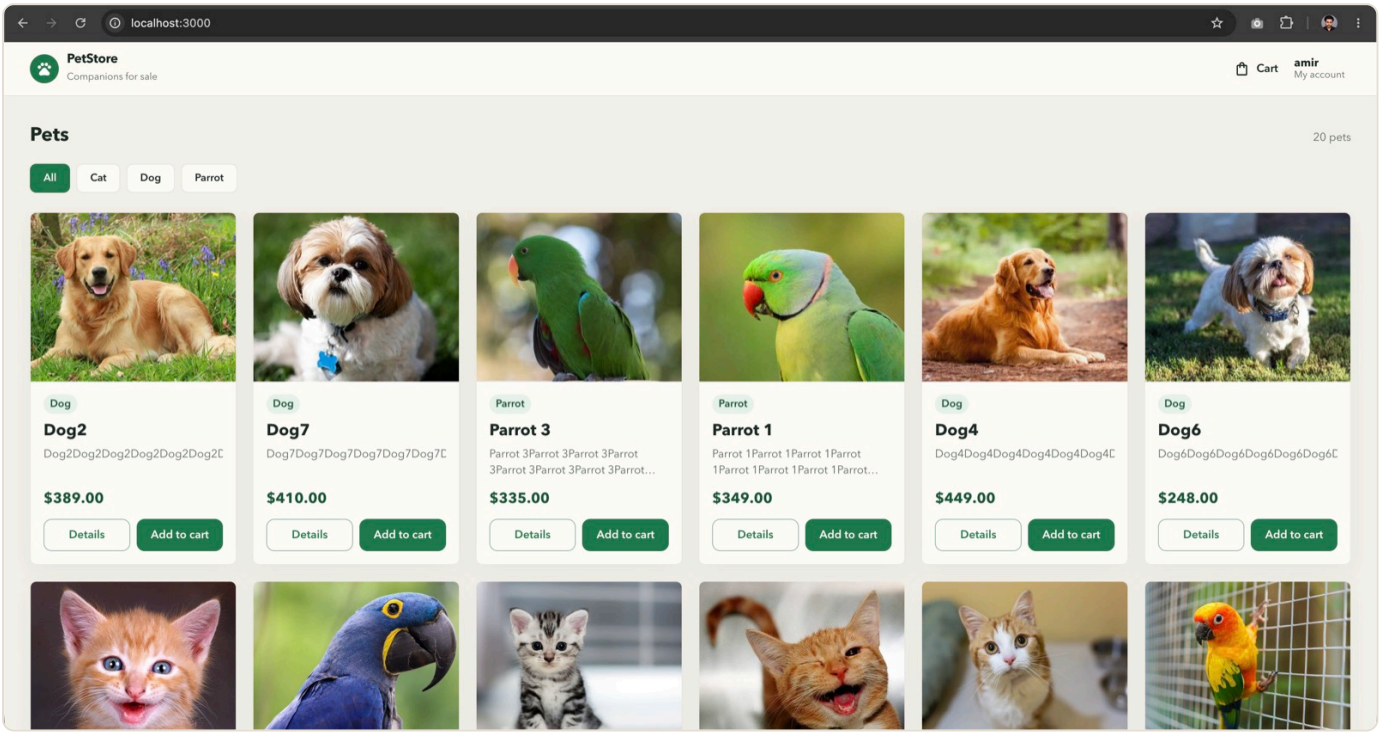


Figure 1. Storefront at `localhost:3000`. Twenty pets, category filters, and a signed-in shopper named amir. The header shows Cart and My account. There is no admin button on the shop.

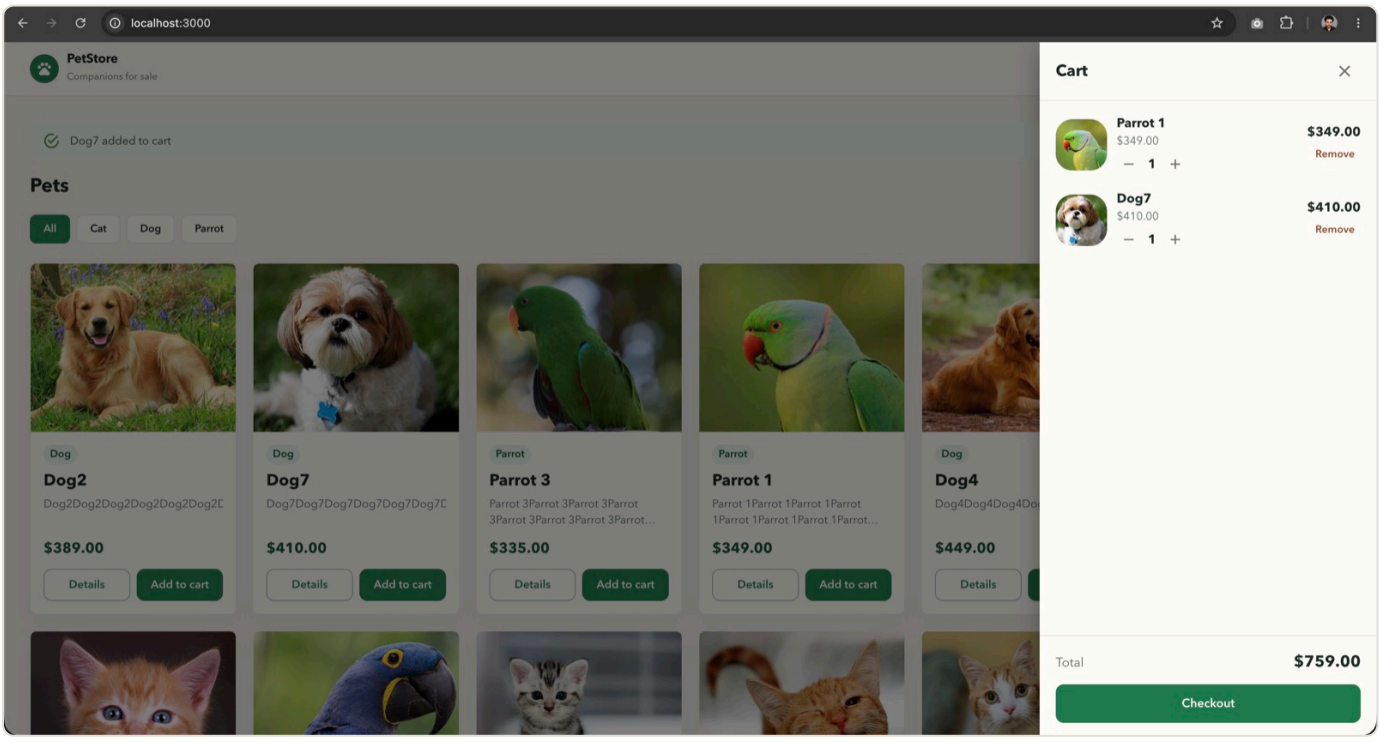


Figure 2. Cart drawer after adding Dog7. Parrot 1 and Dog7 are listed with quantity controls. The total is $759.00 and Checkout stays on this page until the order is placed.

4. Customer orders

A signed-in customer opens My account and sees only their own orders. The page is a data grid with search, column controls, and paging. Columns are Order, Items, Total, Status, and Placed. Customers cannot delete an order from this page. Delete stays on the admin orders screen.

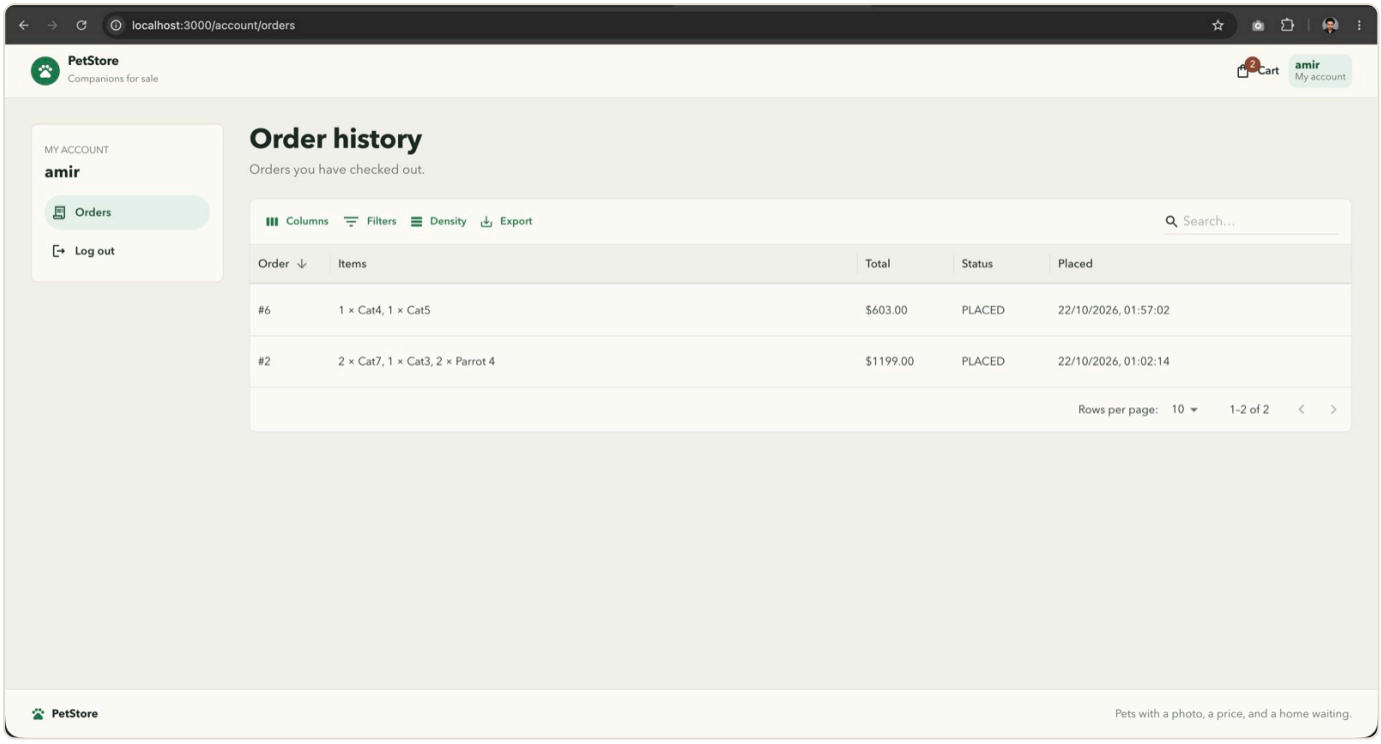


Figure 3. Account page for amir. Order #6 is two cats for $603.00. Order #2 is three pets for $1,199.00. Both are PLACED. Log out is in the account menu, not in the shop header.

5. Administration

Administration lives at `/admin` and uses its own session. The navigation is Categories, Pets, and Orders. The signed-in administrator in these screenshots is `admin`. View store returns to the public shop without clearing the admin session.

Categories

Categories are created here before they can be chosen on a pet. The screenshot shows Cat, Dog, and Parrot, which are the same chips on the storefront. Each row can be deleted. The grid has search, filters, density, and export.

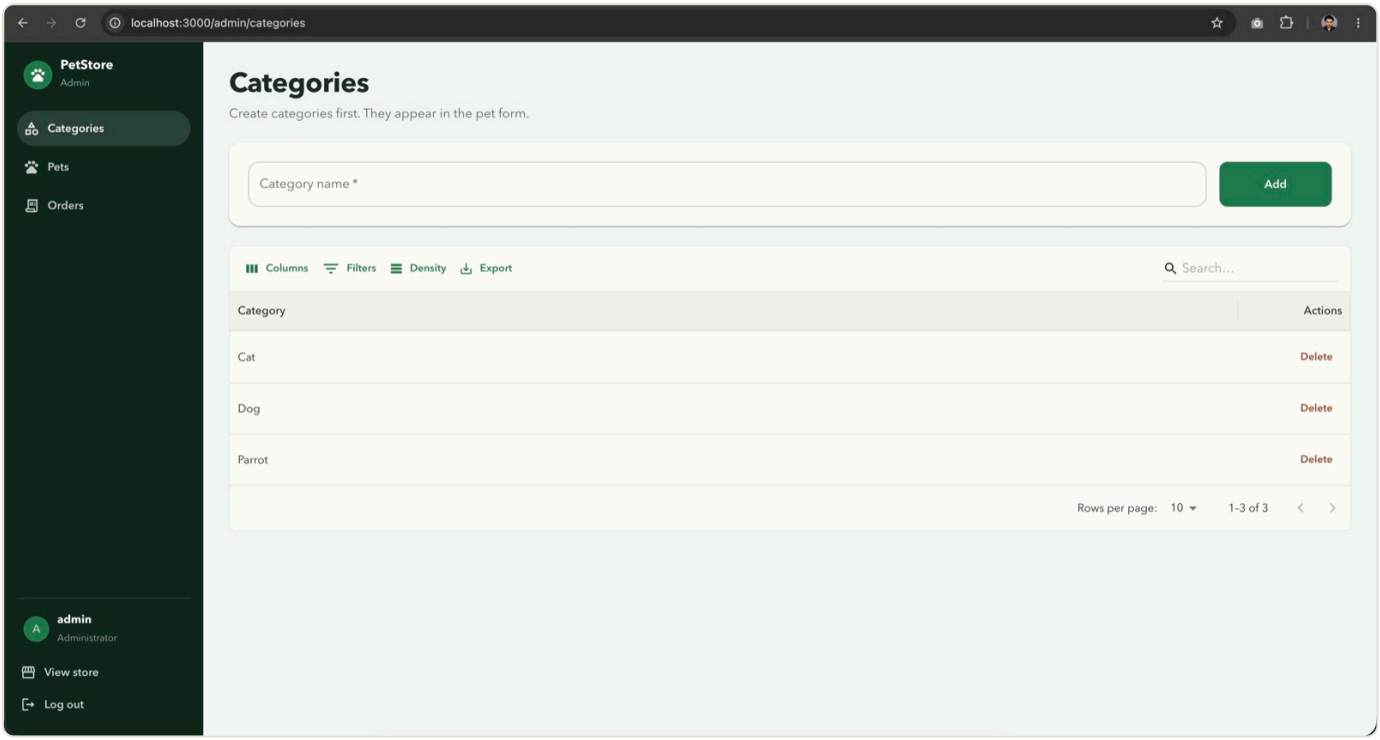


Figure 4. Admin categories. Three categories are in use. The add form is on the same page.

Pets

The pet list is the catalog the shop sells. Columns are Photo, Name, Category, Description, Price, and Actions. Edit and Delete are on each row. A new pet requires a photo. Uploads go to `pet-container`. Replacing a photo deletes the previous blob. Deleting a pet deletes its blob as well.

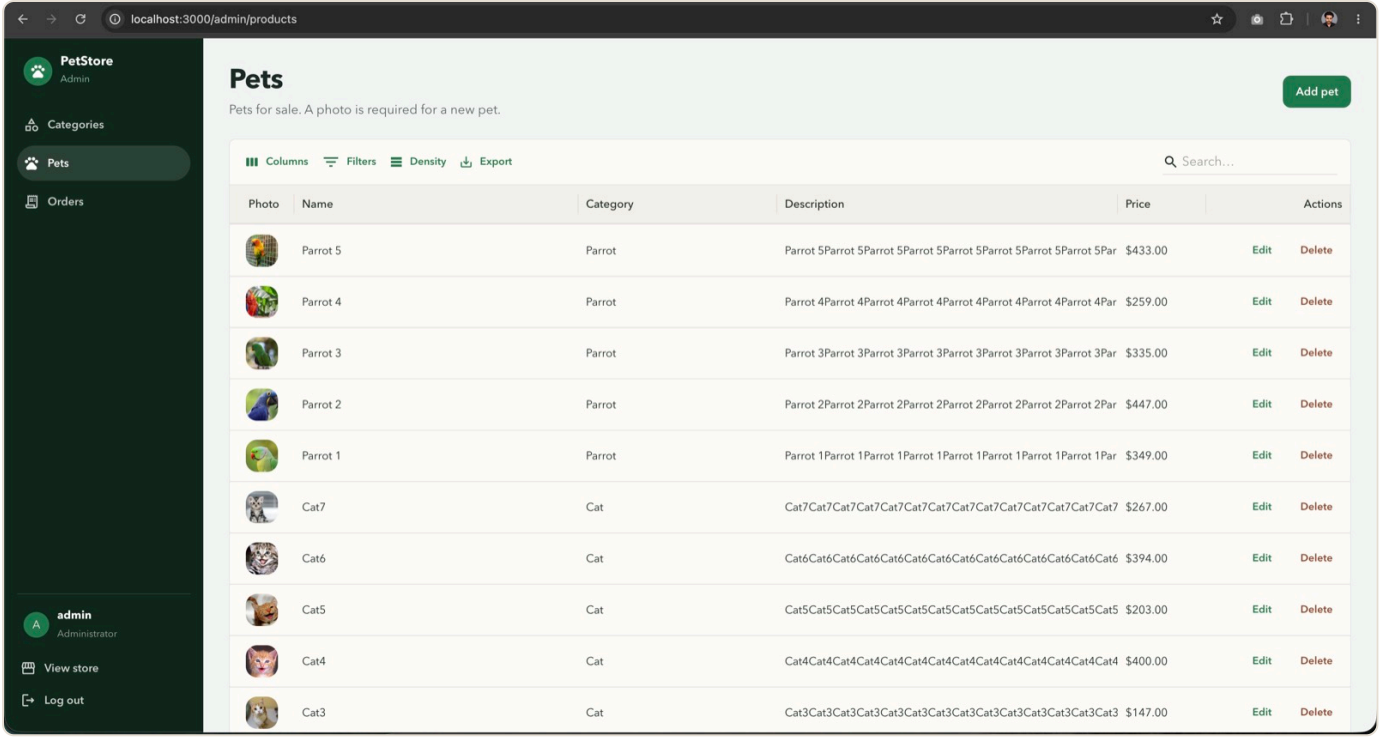


Figure 5. Admin pets. Parrots and cats are listed with the photograph stored in Blob Storage, a category, a price, and Edit / Delete.

Orders

Administrators see every order, including the customer id. The same orders that appear on a customer's account page appear here, with an extra Customer column and a Delete action. Delete is allowed only for the ADMIN role.

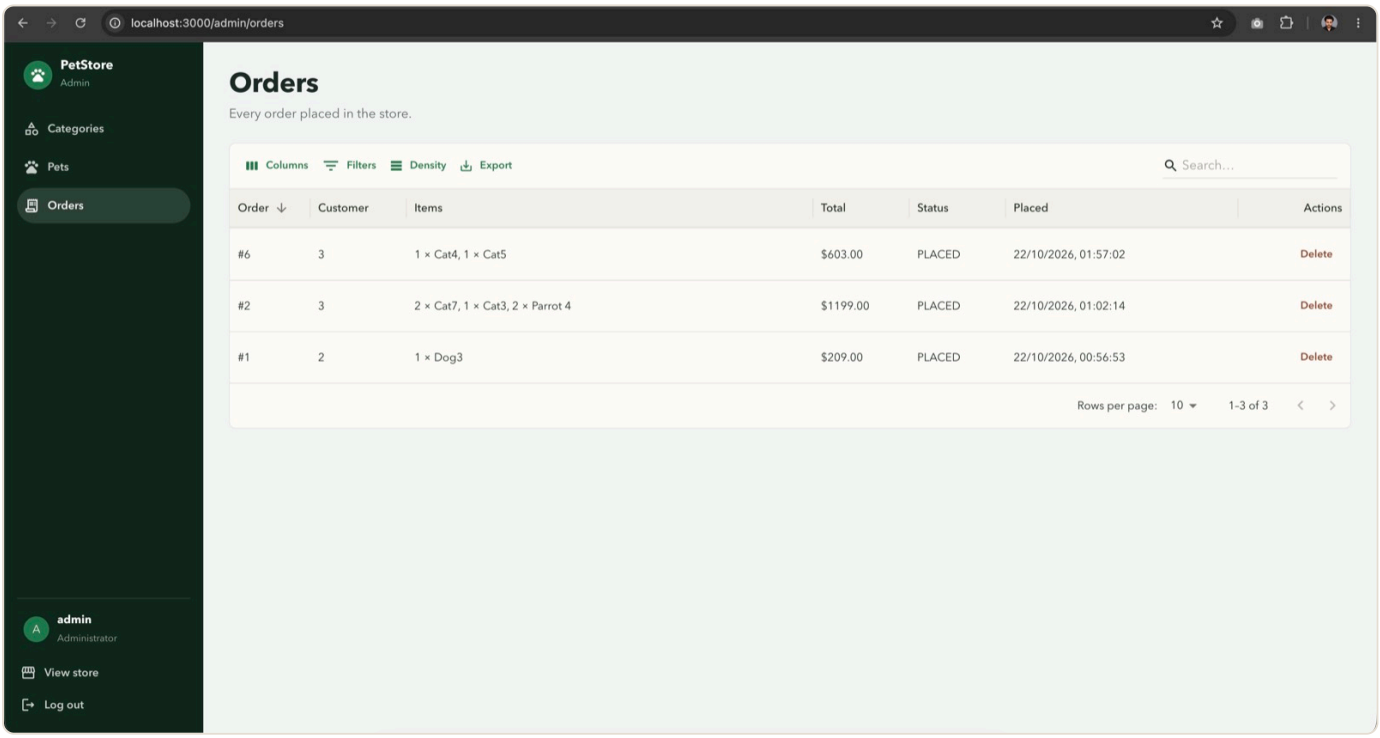


Figure 6. Admin orders. Orders #6 and #2 belong to customer 3. Order #1 belongs to customer 2. Each row can be deleted.

6. Azure Database for PostgreSQL

All three services use one database, `petstore`, on an Azure Database for PostgreSQL flexible server. The server is in the Petstore resource group and is Ready.

Setting	Value
Server	petstore-server-name
Endpoint	petstore-server-name.postgres.database.azure.com
Subscription	Petstore subscription · b5c67bf9-6f36-431b-82ea-a27d96f61190
Resource group	Petstore-Resource-Group
Location	Central India
PostgreSQL	16
Compute	Burstable B2s · 2 vCores · 4 GiB RAM · 32 GiB storage
Administrator login	amirrahi29
High availability	Not enabled. This tier is aimed at development and test.

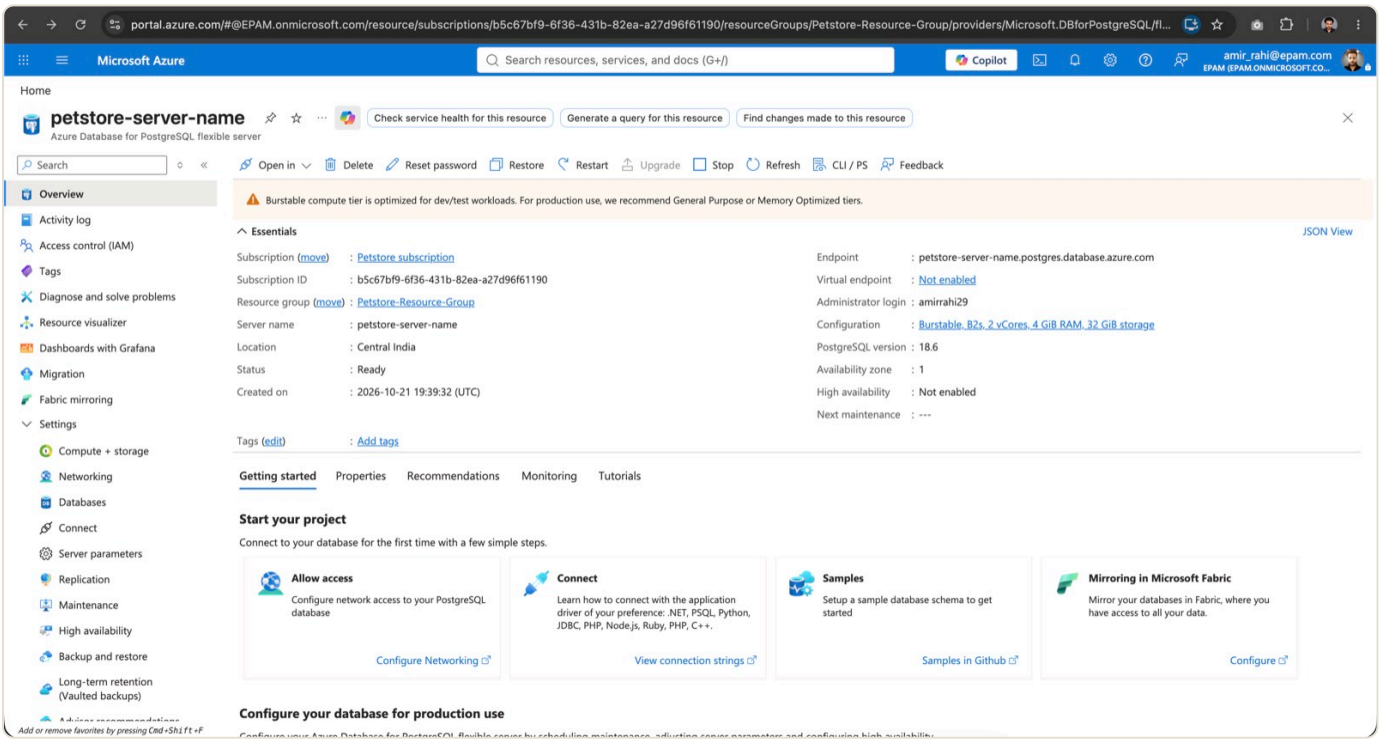


Figure 7. Flexible server overview. Status is Ready. The JDBC URL in each service points at this endpoint and the `petstore` database.

pgAdmin is connected to that server as `amirrahi29`. The `public` schema holds the application tables: `categories`, `order_items`, `orders`, `pets`, `products`, `roles`, and `users`. The products query in the screenshot returns the same catalog the storefront shows, twenty rows, with category and description filled in.

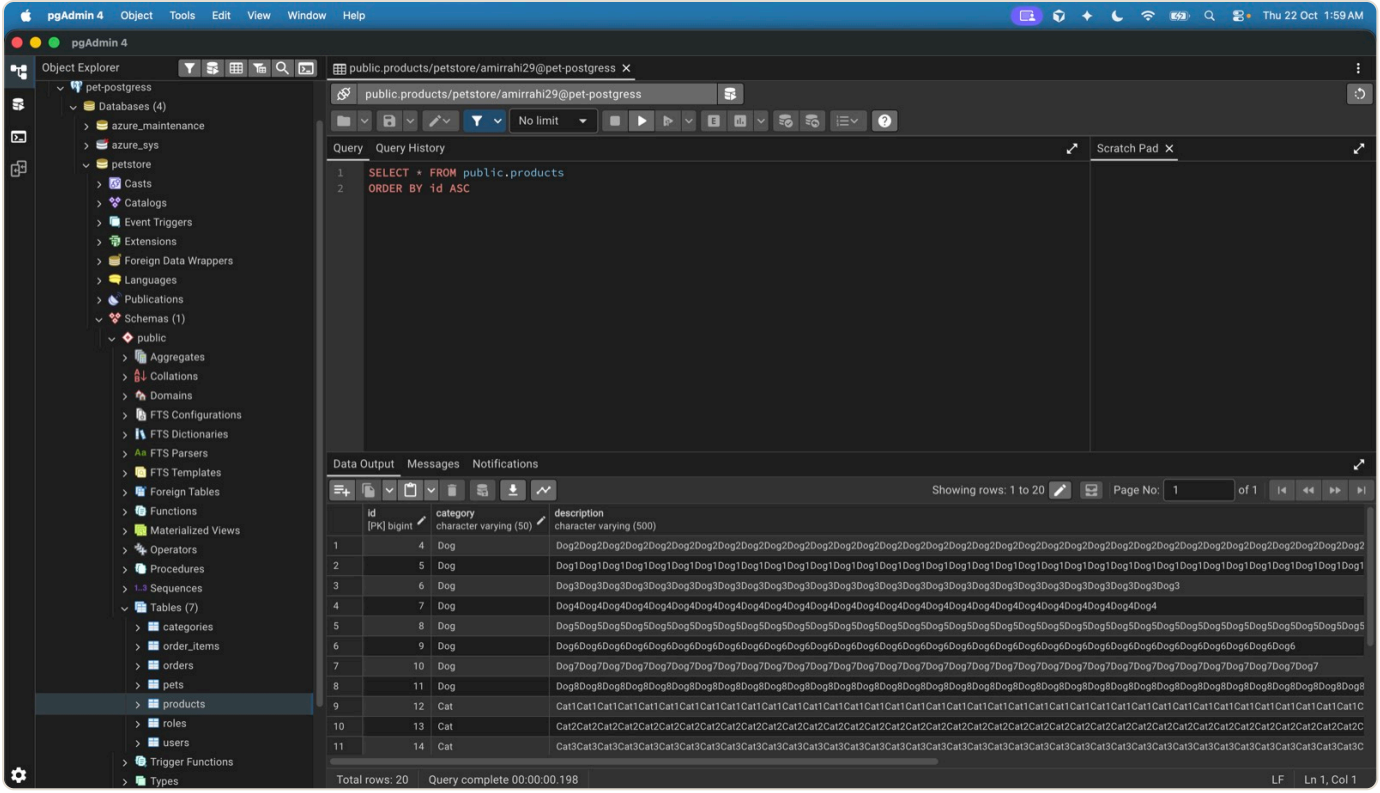


Figure 8. Database `petstore` on server pet-postgres. `SELECT * FROM public.products ORDER BY id` returns the pets for sale.

> The database username and password are not written in the service YAML files. Each service reads the Key Vault secrets named `username` and `password` when it starts, then uses them as `spring.datasource.username` and `spring.datasource.password`.

7. Azure Blob Storage

Pet photographs are files in the storage account `petstore`. The application container is `pet-container`. A second container, `$logs`, is the storage log container and is not used by the shop.

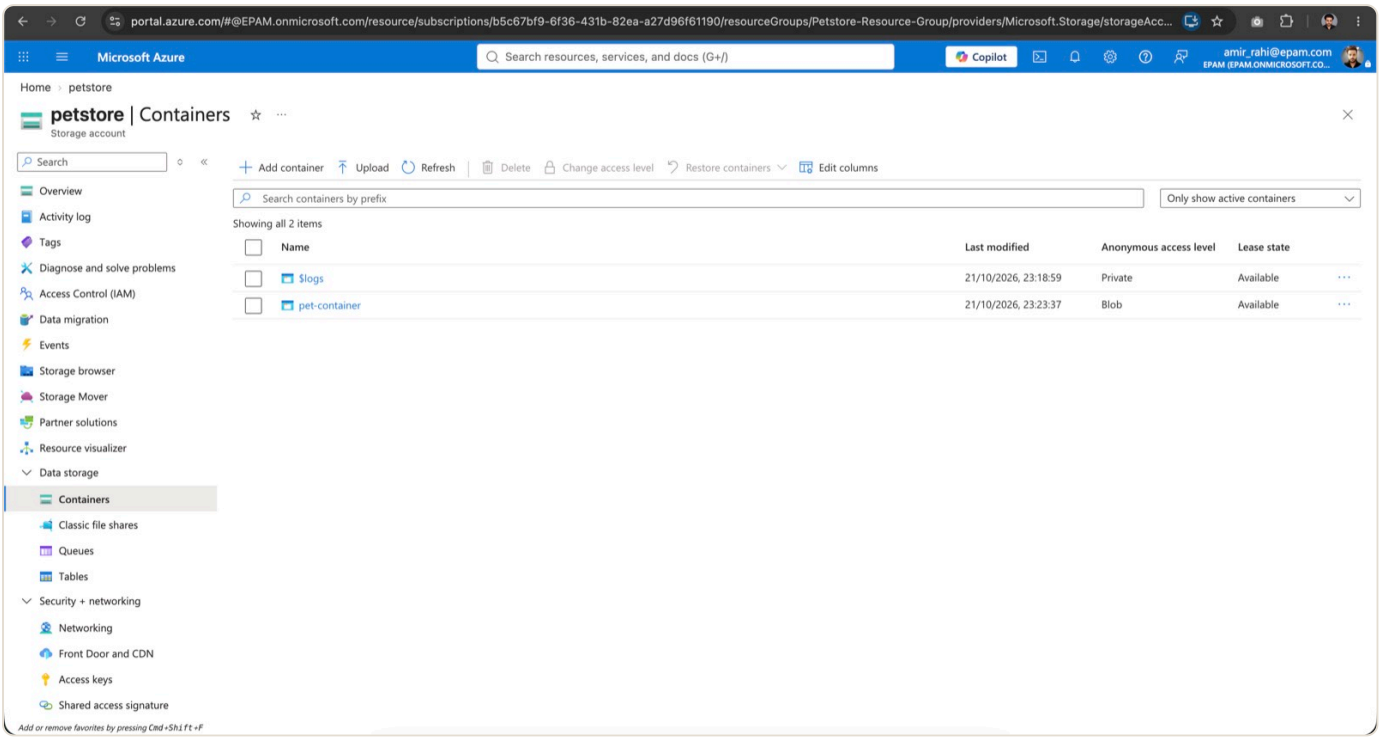


Figure 9. Containers on storage account petstore. `pet-container` is the container named in the product service. Its anonymous access level in the portal is Blob.

Each upload is stored under a generated file name. The database keeps the blob URL. When the API returns a pet, it adds a short-lived shared access signature so the page can show the image. The expiry configured for that link is 60 minutes. The storage connection string and account key come from the Key Vault secrets `connection-string` and `account-key`. The account name, blob endpoint, container name, and SAS lifetime stay in configuration because they are not secret values.

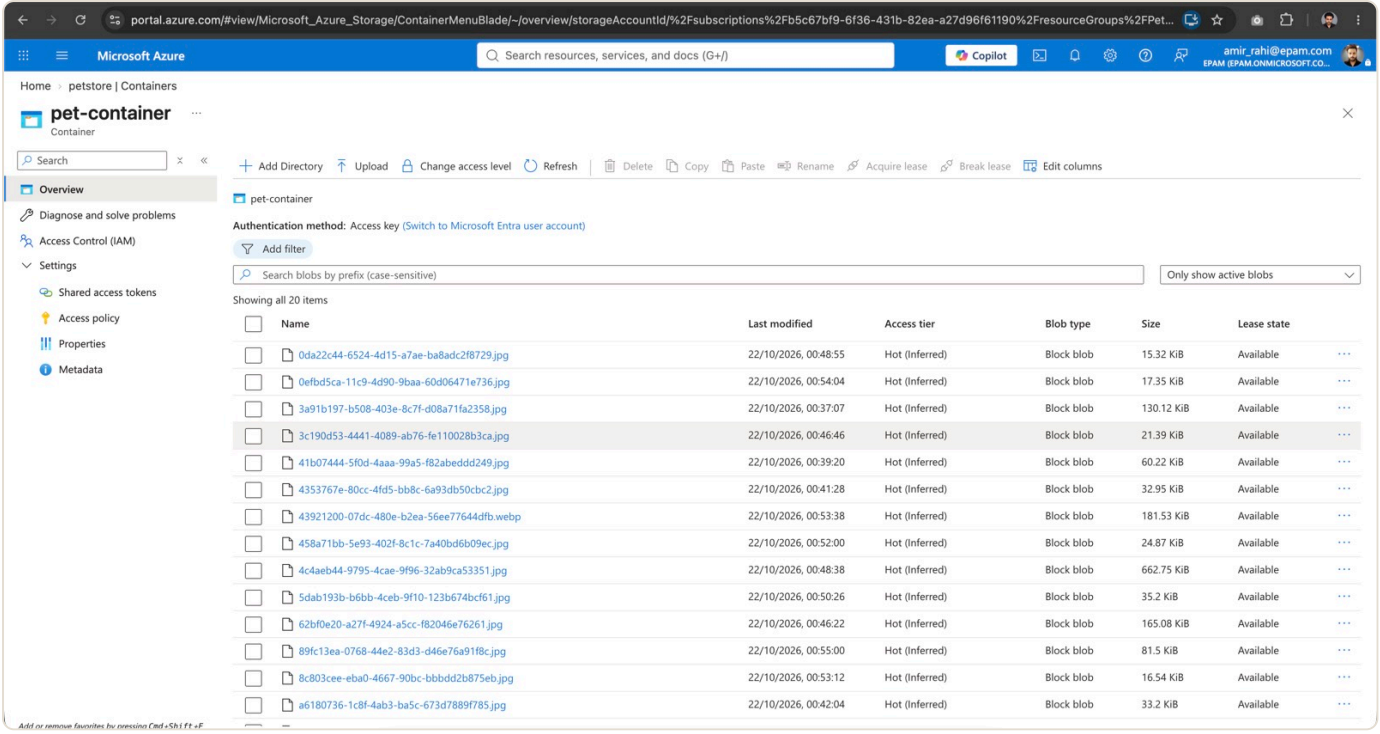


Figure 10. `pet-container` holding the pet images. Twenty blobs are listed, created when pets were saved from the admin form.

8. Azure Key Vault

One vault holds the values the services used to keep in YAML. It is `db-credentials-petstore`, in Petstore-Resource-Group, location India South Central, on the Petstore subscription.

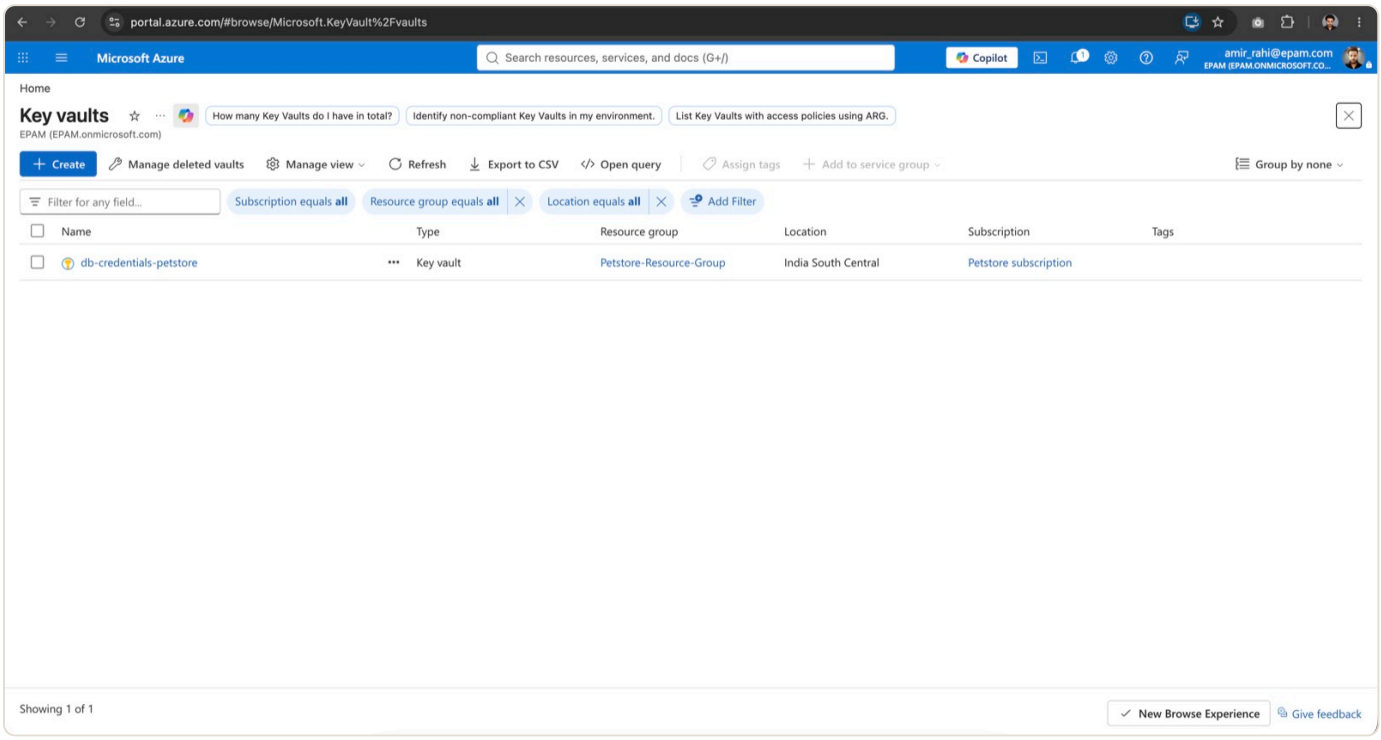


Figure 11. The vault in the subscription. One vault is in use for this project.

Four secrets are enabled. None of the secret values are stored in the repository.

Secret	Used as	Services
`username`	Database username	product, order, pet
`password`	Database password	product, order, pet
`connection-string`	Blob Storage connection string	product
`account-key`	Blob Storage account key	product

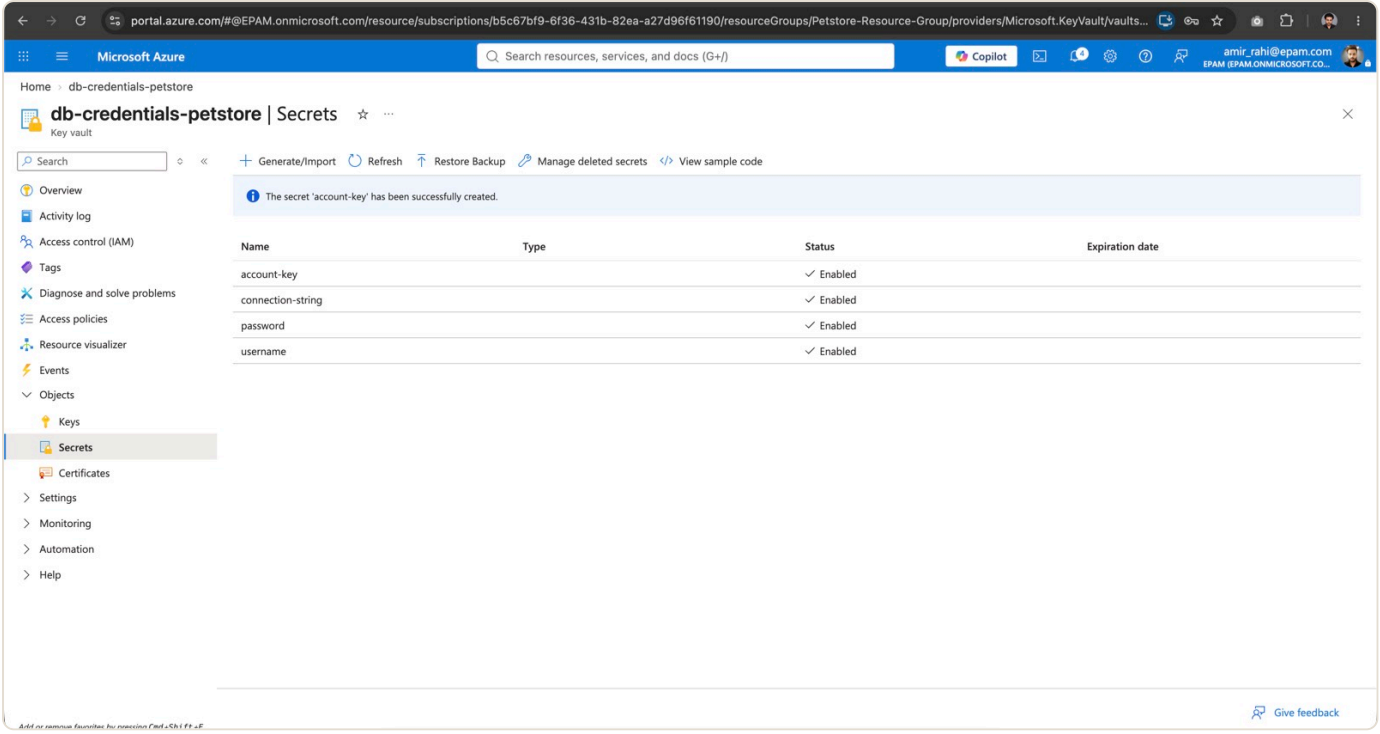


Figure 12. Secrets on `db-credentials-petstore`. All four are Enabled. The portal confirms `account-key` was created.

How a service reads them

The vault URL is the only Key Vault setting in configuration: `https://db-credentials-petstore.vault.azure.net/`. At startup each service signs in with `DefaultAzureCredential` and copies the secret values into the Spring environment before the database connection is opened. On a developer machine that credential is an Azure CLI login for an account that has Get permission on secrets. No client secret is written into the YAML.

> A service that starts without an Azure login stops with a clear error: it could not read the secrets from this vault. It does not fall back to a password in a file. Sign in with `az login` as a principal that can get secrets, then start the jars again.

9. Azure Container Registry

Each application already has a Dockerfile. A push to the `main` branch of `amirrahi29/Petstore-Microservice-App-SpringBoot-React` builds all four images and stores them in Azure Container Registry. The registry login server is `petstoreazurecontainerregistry.azurecr.io`.

Setting	Value
Registry	petstoreazurecontainerregistry
Login server	petstoreazurecontainerregistry.azurecr.io
Resource group	Petstore-Resource-Group
Subscription	Petstore subscription · b5c67bf9-6f36-431b-82ea-a27d96f61190
Location	India South Central
SKU	Basic. Provisioning succeeded on 22 October 2026.
Sign-in from GitHub	OpenID Connect. The workflow does not store a registry password.

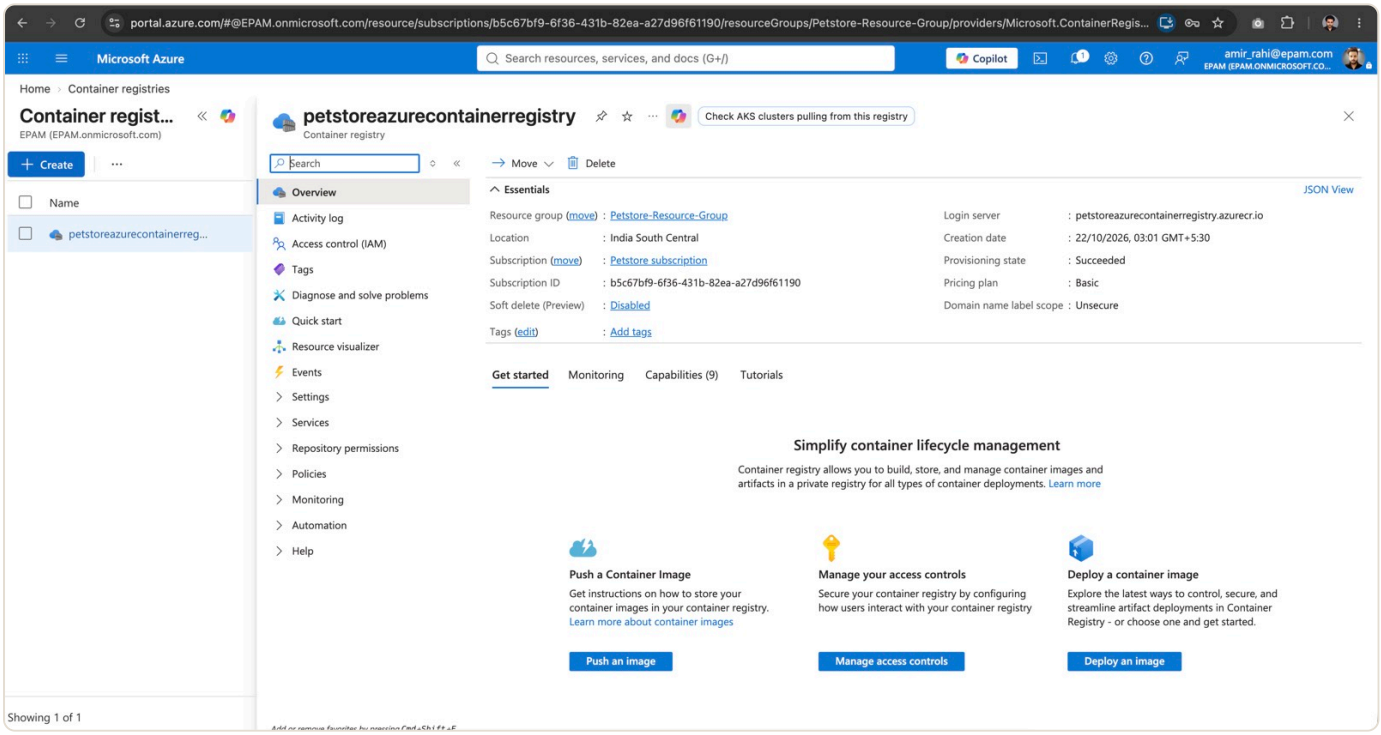


Figure 13. Registry overview. One registry is in the Petstore resource group, on the Basic plan, in India South Central.

The repository name is the service name with `-image` on the end. Each repository has two tags: `latest`, and the git commit that built it.

Application	Image	Port inside the image
product-service	product-service-image	8081
order-service	order-service-image	8083
pet-service	pet-service-image	8082
petstore-ui	petstore-ui-image	80

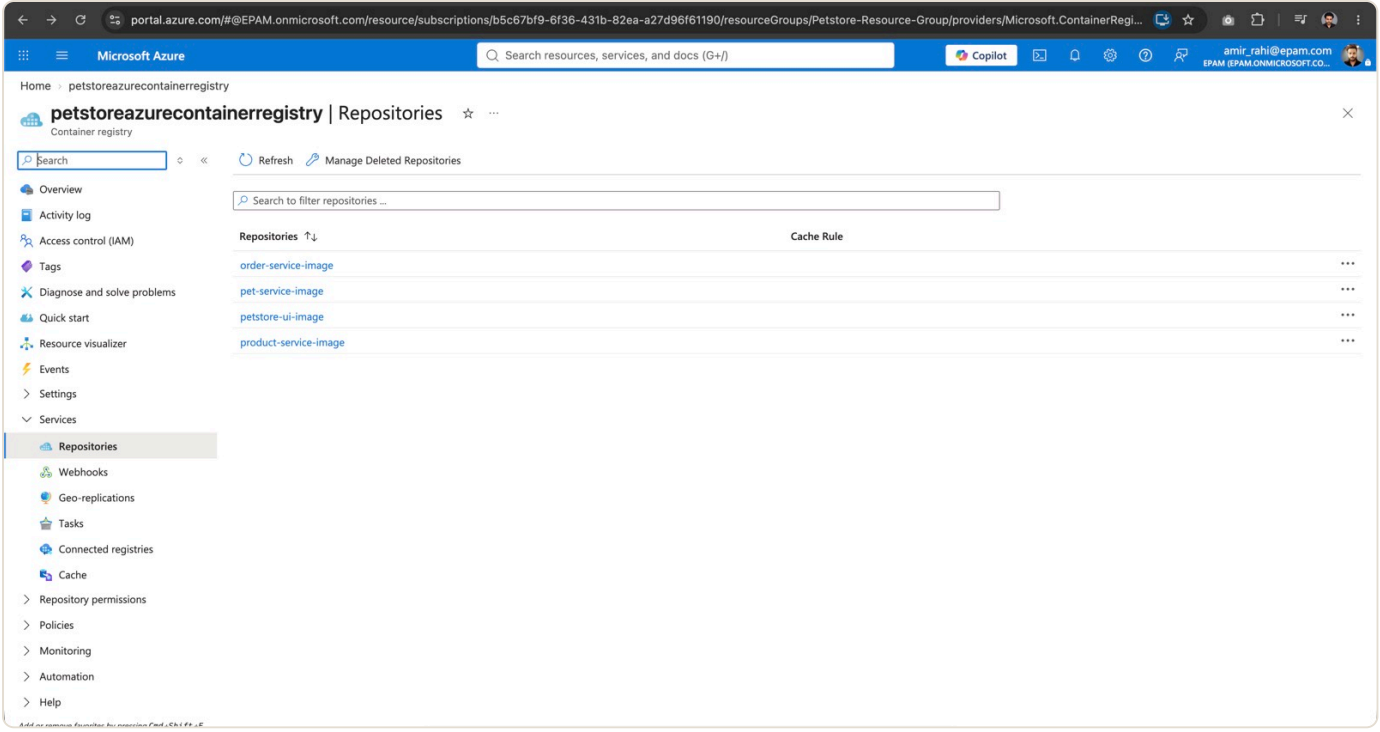


Figure 14. Repositories in the registry: `order-service-image`, `pet-service-image`, `petstore-ui-image`, and `product-service-image`.

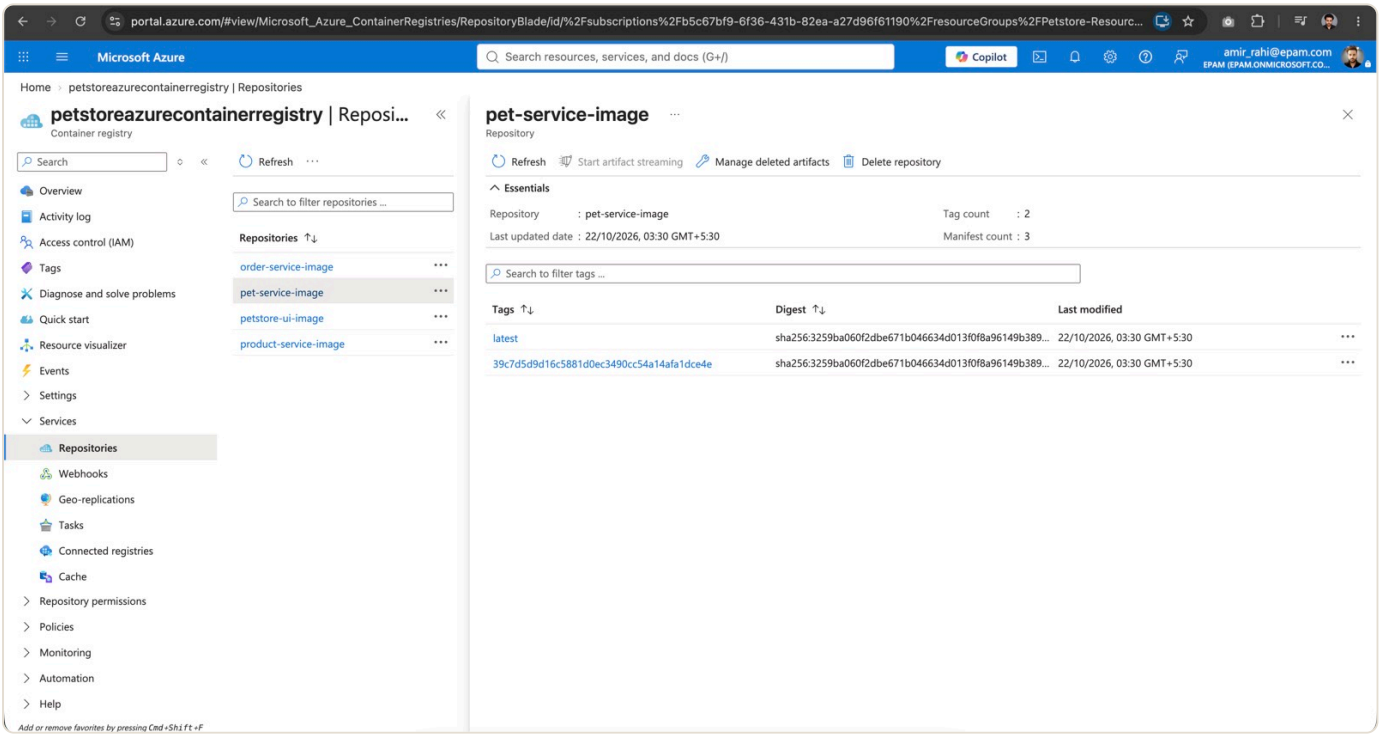


Figure 15. `pet-service-image` has two tags, `latest` and the commit `39c7d5d`. The same pair is published for the other three images.

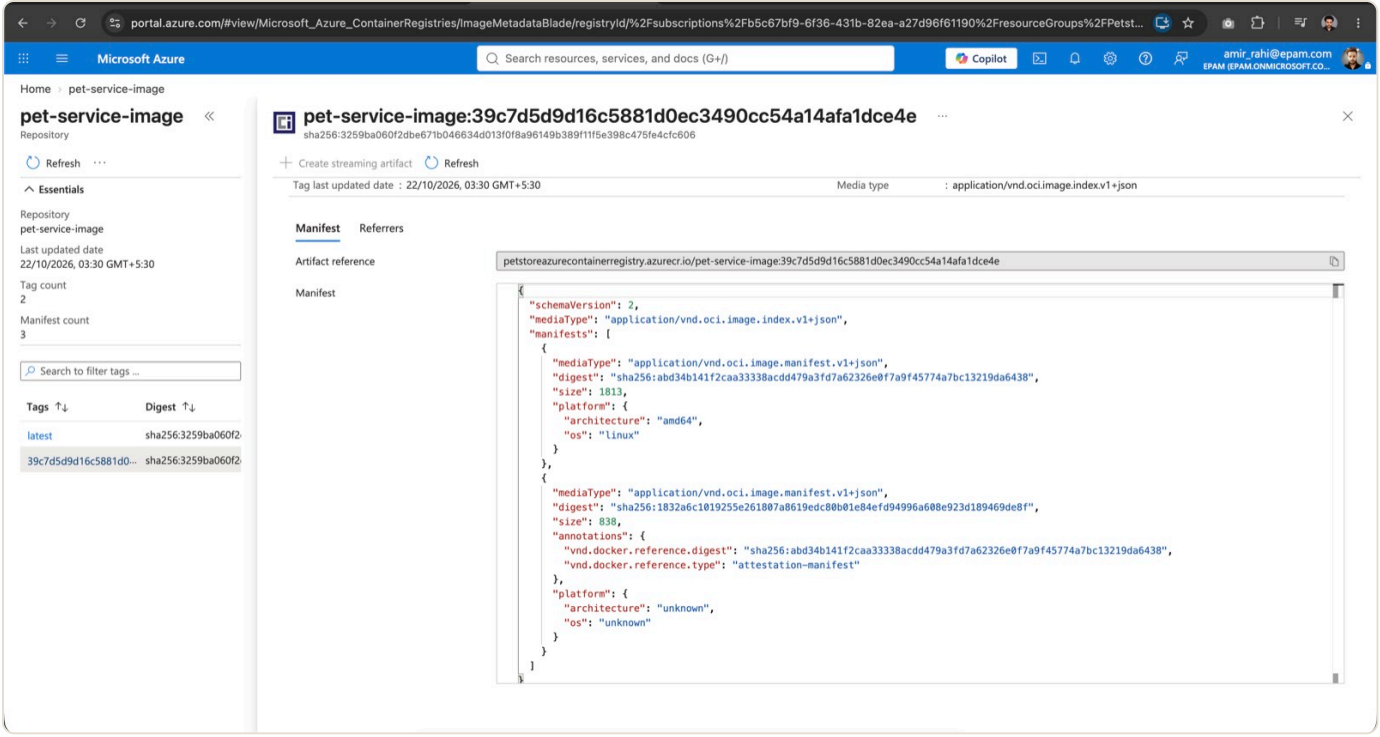


Figure 16. Manifest for `petstoreazurecontainerregistry.azurecr.io/pet-service-image:39c7d5d9d16c5881d0ec3490cc54a14afa1dce4e`. The image is a Linux amd64 OCI index.

> The four workflow files live in `.github/workflows`. A push to `main` starts all of them. While a build is running, GitHub shows a yellow status on the commit. It turns green when that image is in the registry.

10. Run the project

Java 21, Maven, Node 20, and an Azure CLI login are enough on a developer machine. The database and the blob account are already in Azure. From each service directory:

```
java -jar target/product-service-0.1.0.jar --spring.config.additional-location=file:./src/main/resources/application.yml

java -jar target/order-service-0.1.0.jar --spring.config.additional-location=file:./src/main/resources/application.yml

java -jar target/pet-service-0.1.0.jar --spring.config.additional-location=file:./src/main/resources/application.yml
```

The extra configuration location makes the source YAML win over the copy packed inside the jar. The UI is `npm run dev` in `petstore-ui`, which serves `http://localhost:3000`. It calls product-service on 8081 and order-service on 8083.

Screenshot index

File	Figure	What it proves
mainui1.png	1	Public catalog, categories, prices, photos
mainui2.png	2	Cart drawer and checkout
mainui3.png	3	Customer order history
admin1.png	4	Category administration
admin2.png	5	Pet administration with photos
admin3.png	6	All orders, with delete
postgressql1.png	7	Flexible server is Ready
postgressql2.png	8	Tables and product rows
blob1-img.png	9	Storage account and pet-container
blob2-img.png	10	Uploaded pet images
key-vault1.png	11	Vault in the subscription
key-vault2.png	12	The four enabled secrets
container-registry1.png	13	Container registry overview
container-registry2.png	14	The four image repositories
container-registry3.png	15	latest and commit tags
container-registry4.png	16	Image manifest for pet-service-image

Source screenshots remain in the project folder `screenshots/`. This report uses those files. Secret values are not printed.